

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1402

COURSE NAME: Compiler Design for Higher
Level Processing

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS:4

Total Marks:50

Annexure A

DESIGN TASK

Code of Conduct:

I Monika.R(192324281) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Monika.R

Course Faculty

Annexure A

DESIGN TASK

Q2. Shift-Reduce Parsing (Application Level)

Task:

Apply shift-reduce parsing to the given grammar and input string, and demonstrate the complete parsing process by showing each step with stack contents, remaining input, and parsing actions (shift, reduce, accept, or error), clearly indicating all reductions and decisions made during parsing.

Grammar:

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow \text{id}$

Conditions:

- Show stack, input buffer, and action at each step
- Resolve conflicts using precedence (* has higher precedence than +*)

Test Case:

Input:

id + id * id

Expected Output:

- Sequence of actions: Shift / Reduce / Accept
- Final result: **String Accepted**

Integrated Design Question (SLR, LALR, CLR Parsing)

ANSWER:

C program:

```
#include <stdio.h>
#include <string.h>
char stack[100] = "";
char input[100];
int top = -1;
void push(char *str)
{
    int i;
    for(i = 0; str[i] != '\0'; i++)
        stack[++top] = str[i];
    stack[top + 1] = '\0';
}
void reduce()
{
    int len = strlen(stack);
    if(len >= 2 && stack[len-2]=='i' && stack[len-1]=='d')
    {
        top -= 2;
        stack[top+1]='\0';
        push("E");
        printf("%-20s %-20s Reduce E->id\n", stack, input);
        reduce();
        return;
    }
    len = strlen(stack);
    if(len >= 3 && stack[len-3]=='E' && stack[len-2]=='*' && stack[len-1]=='E')
```

```

{
    top -= 3;
    stack[top+1]='\0';
    push("E");
    printf("%-20s %-20s Reduce E->E*E\n", stack, input);
    reduce();
    return;
}
len = strlen(stack);
if(len >= 3 && stack[len-3]=='E' && stack[len-2]=='+' && stack[len-1]=='E')
{
    if(strchr(input, '*') != NULL)
        return;
    top -= 3;
    stack[top+1]='\0';
    push("E");
    printf("%-20s %-20s Reduce E->E+E\n", stack, input);
    reduce();
    return;
}
}
int main()
{
    strcpy(input, "id+id*id");
    printf("\nShift Reduce Parsing\n");
    printf("-----\n");
    printf("%-20s %-20s %s\n", "Stack", "Input", "Action");
    printf("-----\n");
    while(strlen(input) > 0)
    {
        if(input[0]=='i' && input[1]=='d')
        {
            push("id");
            memmove(input, input+2, strlen(input)-1);
            printf("%-20s %-20s Shift id\n", stack, input);
            reduce();
        }
        else
        {
            char temp[2];
            temp[0]=input[0];
            temp[1]='\0';
            push(temp);
            memmove(input, input+1, strlen(input));
            printf("%-20s %-20s Shift %s\n", stack, input, temp);
            reduce();
        }
    }
    reduce();
    printf("-----\n");
    if(strcmp(stack, "E")==0)
        printf("Result : String Accepted\n");
    else
        printf("Result : String Rejected\n");
    return 0;
}

```

OUTPUT:

```
C:\Users\monik\Documents\2 × + ▾ - □ ×

Shift Reduce Parsing
-----
Stack          Input          Action
-----
id             +id*id         Shift id
E              +id*id         Reduce E->id
E+             id*id          Shift +
E+id           *id            Shift id
E+E            *id            Reduce E->id
E+E*           id             Shift *
E+E*id         Shift id
E+E*E          Reduce E->id
E+E            Reduce E->E*E
E              Reduce E->E+E
-----
Result : String Accepted

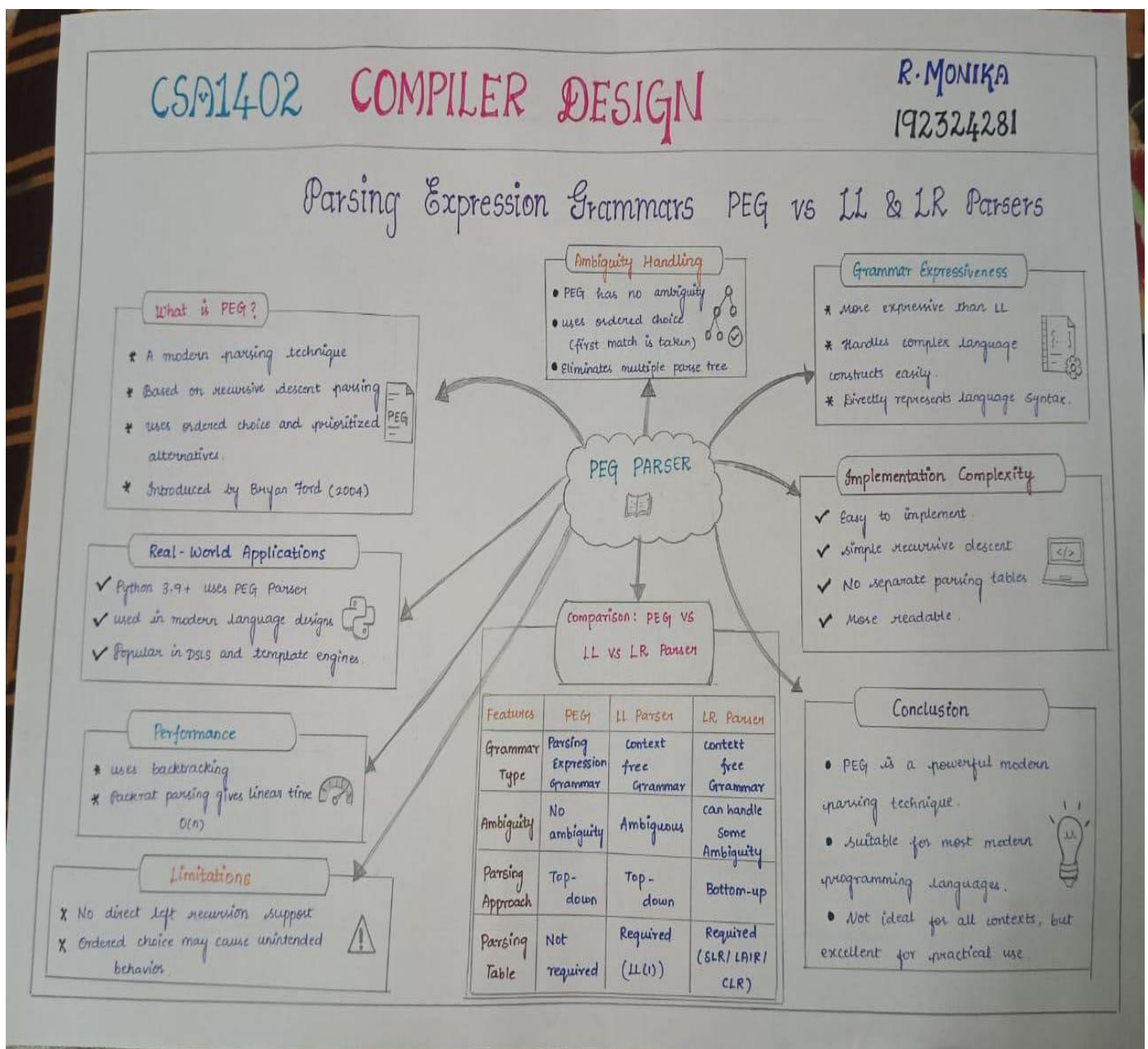
-----
Process exited after 0.0256 seconds with return value 0
Press any key to continue . . . |
```

Q4. Critically evaluate the effectiveness of Parsing Expression Grammars (PEG) as a modern parsing technique by comparing them with traditional LL and LR parsers in terms of ambiguity handling, grammar expressiveness, implementation complexity, performance, and real-world applicability.

Your answer should:

1. **Analyze** how PEG parsers resolve ambiguity differently from CFG-based parsers
2. **Compare** PEG with LL and LR parsing in terms of:
 - o Grammar expressiveness
 - o Ease of implementation
 - o Performance and backtracking
3. **Examine** real-world adoption (e.g., Python's PEG parser) and justify why PEG was preferred
4. **Critically evaluate** limitations of PEG (e.g., lack of left recursion support, ordered choice behavior)
5. **Conclude** whether PEG is suitable for all programming languages or only specific cases, with justification

ANSWER:



Q7. Operator Precedence Parsing Design and Evaluation

Task:

Design an operator precedence parser and evaluate its correctness for handling precedence and associativity.

Grammar:

$E \rightarrow E + E \mid E * E \mid (E) \mid id$

Conditions:

- Define operator precedence relations ($<$, $=$, $>$)
- Construct precedence table
- Handle associativity rules explicitly
- Detect and resolve conflicts
- Show step-by-step parsing actions
- Evaluate correctness of parsing decisions

Test Case:

Input: $id + id * id$

ANSWER:

C program

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>

#define MAX 100

char stack[MAX];

char input[MAX];

int top = -1, i = 0;

int precedence(char op)
{
    switch(op)
    {
        case '+': return 1;

        case '*': return 2;

        case '(': return 0;

        case '$': return -1;
```

```
    }

    return -1;

}

int isOperator(char c)

{

    return (c=='+' || c=='*' || c=='(' || c==')' || c=='$');

}

void push(char c)

{

    stack[++top] = c;

    stack[top+1] = '\0';

}

char pop()

{

    return stack[top--];

}

char topTerminal()

{

    int j = top;

    while(j >= 0)

    {

        if(isOperator(stack[j]))

            return stack[j];

        j--;

    }

    return '$';

}
```

```

int main()

{

    printf("Enter expression (Example: i+i*i): ");

    scanf("%s", input);

    strcat(input, "$");

    push('$');

    printf("\n%-20s %-20s %-15s\n", "Stack", "Input", "Action");

    printf("-----\n");

    while(1)

    {

        char a = topTerminal();

        char b = input[i];

        if(b == 'i')

        {

            printf("%-20s %-20s Shift i\n", stack, &input[i]);

            push('i');

            i++;

            pop();

            push('E');

            printf("%-20s %-20s Reduce i -> E\n", stack, &input[i]);

            continue;

        }

        if(a == '$' && b == '$' && top == 1 && stack[1] == 'E')

        {

            printf("%-20s %-20s Accept\n", stack, &input[i]);

            break;

        }

    }

}

```



```
if(b == '(')
{
    printf("%-20s %-20s Shift (\n", stack, &input[i]);
    push('(');
    i++;
    continue;
}

if(b == ')')
{
    while(stack[top] != '(')
        pop();
    pop();
    push('E');
    printf("%-20s %-20s Reduce (E)\n", stack, &input[i]);
    i++;
    continue;
}

if(b == '+' || b == '*')
{
    if(precedence(a) < precedence(b))
    {
        printf("%-20s %-20s Shift %c\n", stack, &input[i], b);
        push(b);
        i++;
    }
    else
    {

```

```
    if(top>=2 && stack[top]=='E' &&
        (stack[top-1]=='+' || stack[top-1]=='*') &&
        stack[top-2]=='E')
    {
        pop();
        pop();
        pop();
        push('E');
        printf("%-20s %-20s Reduce E op E\n", stack, &input[i]);
    }
    else
    {
        printf("Parsing Error!\n");
        return 0;
    }
}

continue;
}

if(b=='$')
{
    if(top>=2 && stack[top]=='E' &&
        (stack[top-1]=='+' || stack[top-1]=='*') &&
        stack[top-2]=='E')
    {
        pop();
        pop();
        pop();
```

```

    push('E');

    printf("%-20s %-20s Reduce E op E\n", stack, &input[i]);

}

else if(top==1 && stack[1]=='E')

{

    continue;

}

else

{

    printf("Parsing Error!\n");

    return 0;

}

}

}

return 0;

}

```

OUTPUT:

```

C:\Users\monik\Documents\2  X + v
Enter expression (Example: i+i*i): i+i*i

Stack          Input          Action
-----
$              i+i*i$          Shift i
$E             +i*i$          Reduce i -> E
$E             +i*i$          Shift +
$E+            i*i$          Shift i
$E+E           *i$          Reduce i -> E
$E+E           *i$          Shift *
$E+E*          i$          Shift i
$E+E*i         $          Reduce i -> E
$E+E          $          Reduce E op E
$E            $          Reduce E op E
$E            $          Accept

-----
Process exited after 12.08 seconds with return value 0
Press any key to continue . . . |

```

Q8. Ambiguity Removal and Parsing Strategy Evaluation

Task:

Analyze the ambiguity in the given grammar, redesign it to remove ambiguity, and evaluate parsing using shift-reduce parsing.

Grammar:

$E \rightarrow E - E \mid E / E \mid \text{id}$

Conditions:

- Identify ambiguity in the grammar
- Redesign grammar using precedence rules
- Justify associativity decisions
- Apply shift-reduce parsing
- Show all stack operations and reductions
- Evaluate correctness of final parse

Test Case:

Input: id - id / id

ANSWER:

```
#include <stdio.h>

#include <string.h>

#define MAX 100

char stack[MAX];

char input[MAX];

int top = -1, i = 0;

void push(char c)
{
    stack[++top] = c;
    stack[top + 1] = '\0';
}

void pop()
{
    if (top >= 0)
        top--;
}
```

```

int precedence(char op)
{
    if (op == '/')
        return 2;

    if (op == '-')
        return 1;

    return 0;
}

void display(const char *action)
{
    printf("%-20s %-20s %s\n", stack, &input[i], action);
}

int main()
{
    printf("\nSHIFT REDUCE PARSER\n");

    printf("-----\n");

    printf("Grammar:\n");

    printf("E -> E-T | T\n");

    printf("T -> T/F | F\n");

    printf("F -> id\n");

    printf("Enter expression (Example: id-id/id): ");

    scanf("%s", input);

    strcat(input, "$");

    push('$');

    printf("\n%-20s %-20s %s\n", "Stack", "Input", "Action");

    printf("-----\n");

    while (1)

```

```

{
    char ch = input[i];
    if (input[i] == 'i' && input[i + 1] == 'd')
    {
        push('I');          // I represents id
        display("Shift id");
        i += 2;
        pop();
        push('E');
        display("Reduce id -> E");
    }
    else if (ch == '-' || ch == '/')
    {
        while (top >= 3 &&
            stack[top] == 'E' &&
            (stack[top - 1] == '-' || stack[top - 1] == '/') &&
            stack[top - 2] == 'E' &&
            precedence(stack[top - 1]) >= precedence(ch))
        {
            pop();
            pop();
            pop();
            push('E');
            display("Reduce E -> E op E");
        }
        push(ch);
        if (ch == '-')

```

```

        display("Shift -");

else

        display("Shift /");

i++;
}

else if (ch == '$')
{
    while (top >= 3 &&
           stack[top] == 'E' &&
           (stack[top - 1] == '-' || stack[top - 1] == '/') &&
           stack[top - 2] == 'E')
    {
        pop();

        pop();

        pop();

        push('E');

        display("Reduce E -> E op E");
    }

    if (top == 1 && stack[0] == '$' && stack[1] == 'E')
    {
        display("Accept");

        printf("\n-----\n");

        printf("Parsing Successful\n");

        printf("Correct Parse: id - (id / id)\n");

        printf("-----\n");

        break;
    }
}

```

```

else

{

    printf("\nParsing Error!\n");

    break;

}

}

else

{

    printf("\nInvalid Input!\n");

    break;

}

}

return 0;

}

```

OUTPUT:

```

C:\Users\monik\Documents\2  ×  +  ∨

-----
Grammar:
E -> E-T | T
T -> T/F | F
F -> id

Enter expression (Example: id-id/id): id-id/id

Stack          Input          Action
-----
$I             id-id/id$      Shift id
$E             -id/id$        Reduce id -> E
$E-           -id/id$        Shift -
$E-I          id/id$         Shift id
$E-E          /id$          Reduce id -> E
$E-E/         /id$          Shift /
$E-E/I        id$           Shift id
$E-E/E        $             Reduce id -> E
$E-E          $             Reduce E -> E op E
$E            $             Reduce E -> E op E
$E            $             Accept

-----

Parsing Successful
Correct Parse: id - (id / id)

-----

Process exited after 13.68 seconds with return value 0
Press any key to continue . . . |

```


RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	20	Demonstrates thorough understanding and integrates multiple concepts effectively	Good understanding with proper integration of most concepts	Basic understanding with partial integration	Poor understanding with no integration
Design & Implementation	25	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate construction of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis
Documentation & Presentation	20	Well-structured documentation with diagrams, steps, and clarity	Organized documentation with required details	Basic documentation with missing details	Poor or incomplete documentation